

Mycelic research handoff — 2026-10-09

Oct 9, 2026 · @Ali Novruzov

Bottom line

The theory has not failed. It has not been tested yet. All five real-data replays ran lexical word-matching and model-free count detectors. None of them ran a small model, Tesseract, NeuralGraph, the Mycelic fabric or the questioning step.

The test we did run was biased toward a negative.

- The chance baseline counted each outcome's own after-the-fact alerts. That inflated the expected recall finds from 20.30 to 38.64 (offline diagnosis, the three channels pooled).
- At the complaint volume our own diagnostic measured, the state-split detectors had no power: 0 of 270 planted pre-event ramps raised an alert (another simulation). The recorded power check agrees: X, S and R_mf found 0 of 30, and the best channel, PRR, 6 of 30 (`POWER.md` run 002).
- The data window cut off 68–81% of the complaints that came before each event.

Correcting these does not produce a positive. Re-scored in CI with the corrected baseline (DIAG-E002), X found 14 recalls against 16.37 expected and 6 investigations against 5.64; S and R_mf are alike. The finds sit at chance against that baseline, whose own bias runs in an unknown direction, and show no early warning. So the honest verdict is "uninformative", not "refuted".

The first real-data reading test failed the same way. In reader-001, three small models read 150 real complaints and every one scored predicate F1 0.000, against 0.434 for word-matching. Our post-processing discarded each answer that typed the vehicle without resolving it to a pack id. The instrument measured itself, not the models.

Tesseract ingestion works, but nothing downstream uses it. All 5,882 of 5,882 LoCoMo messages became nodes. Tesseract is wired to nothing in Mycelic. Before the fixes it crashed on SQLite, the only persistent storage, and ranked a message's exact text first only 72% of the time. After them (merge 61c893e) it runs on SQLite and ranks that text first in 599 of 600 queries.

The night plan below fixes the instruments first, then puts Tesseract on the path, then re-tests.

Update after the night's runs (22:30 UTC). With the reading instrument fixed, the small models were tested twice on the same 150 real complaints. Both results are real negatives for site models on public text:

- **Open reading (R002, RESULTS run 7):** predicate F1 0.026, 0.178 and 0.218, against 0.434 for word matching.
- **The narrow question HQ actually asks a site (J001, RESULTS run 9):** balanced accuracy 0.500, 0.477 and 0.387, against 0.670 for the lexical judge. The largest model tells filed from unfiled components but answers "unclear" instead of "no" (decision 9).

Detection stays uninformative rather than refuted: with the corrected baseline the finds sit at chance, and the power check shows the detectors could not see a signal of the measured size (`DIAG-E002.md` , `POWER.md`). The leakage scan with a small model in the loop passed (RESULTS run 8).

Generalization, measured (10 Oct, 04:43 UTC). A pack drafter (`mycelic/collective/onboard/`) drafts a pack from an export alone, learning its vocabulary from the export's own filed categories, with no code per field. Its pre-registered test D002 passed all six criteria on public data:

- **Mine accidents (MSHA, five operators, a field no pack covered):** micro F1 0.559 [0.532, 0.586] on held-out 2022-2024 records, against 0.344 for the best control.
- **Vehicle complaints (six makes):** 0.590 against 0.406 for the hand-built `pack/` .

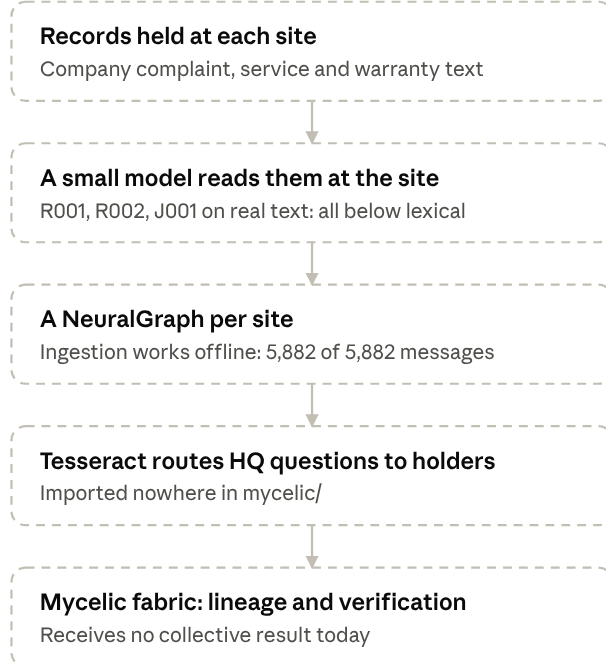
The first rule, D001, crashed on MSHA's quoted fields before reading anything; that is its recorded result (`CHOICE-D001.md` , `CHOICE-D002.md`). This is lexical reading against filed categories, not detection.

What the tests actually ran

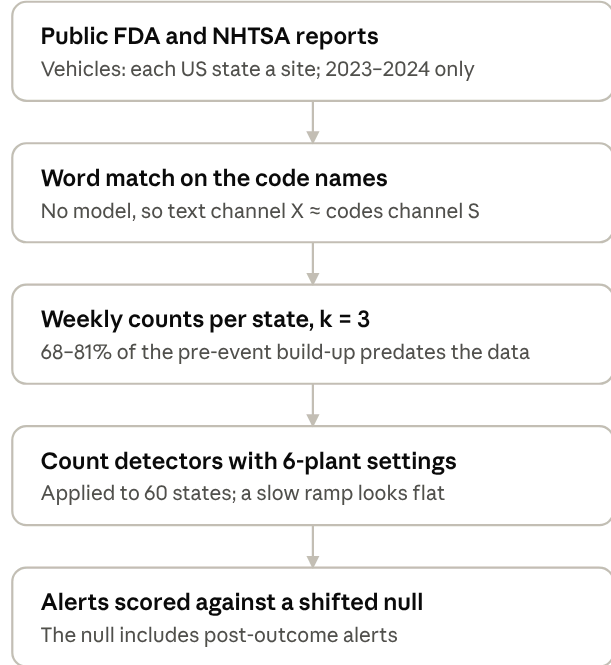
The replays ran a path that skips every part of the theory

Dashed: never in any real-data replay. Solid: what all five device and vehicle replays ran.

The theory



What the five replays ran



intended stack vs tested path · 5 steps each

Every replay negative comes from the right-hand path. No part of the left-hand path, the theory, has met real data in a replay. Only the small-model step has met real text since (R001, R002, J001), and it scored below word matching.

The mistakes we made

The "stupid" mistake is the first row: we kept testing a counting pipeline and reading each negative as a verdict on the theory. Rows 2-7 are why that pipeline could not pass even on data with a real signal. Six investigators found these, and independent reviewers re-checked them against the code and the saved audits.

#	What we did wrong	What it did to the result	Evidence	Where
1	Tested a pipeline with no model, Tesseract, NeuralGraph, fabric or questioning step	The negatives say nothing about the theory either way	0 Tesseract imports in mycelic/ ; every replay runs runtime=None , extract("lexical") ; guard tests forbid adding it	evaluate/baselines.py:134,139 ; test_collective_guards.py:601
2	The chance baseline includes each outcome's own post-outcome alerts	"Fewer finds than chance" is an artifact of complaints reacting to recalls	Recalls: expected 38.64, or 20.30 without them. Investigations: 10.38, or 6.95	openfda_replay.chance_null via pilot/audit.py score_channel
3	No power check before the replays	The test could not pass at the signal size we later measured	0 of 270 planted ramps alerted at 0.1 per week (another simulation); POWER.md run 002 agrees, X, S and R_mf 0 of 30. At a 26-week look-back no channel reaches 80% even at 2.0 a week (best: R_mf 20 of 30). At 104 weeks with 4 years of history, PRR passes at 0.5 a week, not at 0.1	Another simulation, not recorded in the repo; POWER.md run 002
4	Made each US state a "site" but kept the 6-plant detector settings	Noise alerts plus a 4-week cooldown lock out rising series	Synthetic, 60 sites against 6: null candidate rate per test for X 26.15% against 0.38% (PACK-V2.md). A sextupling is caught 8 of 60 times split by state (X, S; R_mf 10), 36 of 60 pooled (POWER.md). The first figures, 43%, 1.8% and 90% pooled, came from a simulation not in the repo and were not reproduced	detect/detectors.py ; vehicle detectors.json is the device pack's file, byte for byte

#	What we did wrong	What it did to the result	Evidence	Where
5	A burst detector for a slow ramp	A 4× rise over 2 years is only 1.25× in the 8-vs-26-week window	DIAG-E001: investigation-matching complaints per quarter go 0.24 → 0.43 → 1.18 → 1.75 (Q-12 to Q-1)	D2 window settings
6	The export kept only 2023–2024	Most of the build-up and many outcomes fell outside the data	68% (recalls) and 81% (investigations) of the prior 3 years' matching complaints predate 2023. Out of scope: 22% of recall and 62% of investigation outcomes	<code>tools/market/nhtsa_export.py</code>
7	Outcomes and complaints use two NHTSA naming schemes	Many outcomes can never match an alert	213 of 896 recall outcomes use the old names; 37% of alerts are on the new names, which match 4 outcomes	<code>tools/market/vehicle_pack.py</code> ; <code>_match_keys</code>
8	The text channel's word list is the code names	X can only re-derive the codes, so $X \approx S$ by construction	2 extra finds for X over S in 12 per-make audits; 30 of 49 X-only patterns are "engine"	vehicle <code>vocabulary.json</code>
9	A missing reporter counts as one shared reporter	The strongest bursts get the "few reporters" penalty	Score 0.3285 → 0.1525 for the same burst	<code>edge/site.py</code> <code>build_cells</code>
10	Model post-processing drops correct answers	A model that follows the prompt scores below the word-matcher	Perfect reader: predicate F1 0.571; with a null vehicle 1.0; lexical 0.588	<code>edge/extract.py</code> : 485–503

#	What we did wrong	What it did to the result	Evidence	Where
11	Tesseract's evidence was leaked or credited to the wrong part	We believed routing worked because of these numbers	Dec-2025 numbers routed on the gold label. The +8.9 gain is speaker-name owner routing, outside <code>tesseract.py</code> ; with it, plain embedding ties or beats Tesseract	<code>research/reports/AUDIT.md:41-44</code> ; <code>H9_two_agent.md:10-15</code>
12	The LoCoMo harness swaps categories 1 and 4	Every "single-hop" and "multi-hop" claim is about the other type	Category 1 averages 3.14 evidence turns (98% multi), category 4 averages 1.07	<code>research/benchmarks/runner.py</code>

Ruled out as the main cause:

- $k=3$ suppression: the unsuppressed `R_mf` channel scored the same as `S` in every make.
- Alerts piling up in the first weeks: 0–8.3% of alert-weeks fall in the first 4 weeks, against 4.6% if spread evenly.

The honest counterweight. Re-scoring the alerts we already have, with the fixes applied, still lands at chance. These are offline diagnosis figures for the union of the three channels, with no recorded run (the recorded per-channel re-score is DIAG-E002, in the bottom line):

- Corrected baseline: 16 found vs 20.30 expected (p 0.58).
- Name crosswalk: 22 vs 25.66 (p 0.60).
- 52-week look-back: 32 vs 29.14 (p 0.43).
- Exploratory make-model match: p 0.068.

One reviewer adds a fair limit: the results do bound any design whose final signal is weekly complaint counts per vehicle and component in 2023–2024.

Tesseract and NeuralGraph ingestion

Ingestion into NeuralGraph storage loses no messages. Retrieval through Tesseract had five defects, two of which blocked a per-site deployment. The table shows the state before the fixes; what changed is under it. The checks below ran offline on all 10 LoCoMo conversations. The real embedding server cannot be reached from this sandbox, so they

used the repo's deterministic hash embedder (`chat_memory/11m.py:201`). The structural results do not depend on the embedder; the recall percentages may change with real embeddings.

Check	Result	Verdict
Messages become nodes	5,882 of 5,882 in 31 s	Works
Session date and YEAR_/MONTH_ tokens on each node	5,882 of 5,882 correct	Works
Tesseract on SQLite, the only persistent store	<code>AttributeError</code> : SQLite storage has no <code>get_neighbors</code>	Broken; blocks per-site use
Exact message text as the query: is that message ranked first?	433 of 600 (72.2%); top 10: 543; outside top 50: 18. Plain cosine: 599 of 600	Broken: all 57 misses name a speaker, and the name match swamps the text
Query-type router	Of 1,536 questions that cite evidence, 895 score highest on "open", which has no store, and 27 fire no marker; 682 of 2,361 gold evidence ids (28.9%) are in no store's candidate list	Broken
Router on cross-site questions ("rising across sites")	No marker fires; near-uniform weights (0.25 / 0.31 / 0.25 / 0.19)	Cannot route
Message dates	<code>created_at</code> is the wall clock (0 of 5,882 carry the message year); the computed date tokens are never read	Broken
No embedding server	Ingestion silently creates 0 of 20 nodes; an empty query embedding raises <code>ValueError</code>	Fails silently

Check	Result	Verdict
dia_id, session index, image captions	Dropped; 37.9% of evidence points at image turns; 37 questions are answerable only from a caption	Missing
LoCoMo category labels	Categories 1 and 4 are swapped in <code>runner.py</code>	Wrong labels
Tests	No test calls Tesseract; the one service integration test is skipped and fakes its score	Untested
Latency per query	272 ms at 600 nodes; 2.6 s at 5,882; 8.7 s at 20,000 (flat scan)	Too slow for a site with tens of thousands of records

After the fixes (merges 61c893e and 921352a, 2026-10-09; hash embedder, 60 messages from each of the ten conversations for the query checks):

- SQLite has `get_neighbors`, and 600 of 600 queries return the same top 10 on SQLite as in memory.
- Exact message text as the query ranks that message first in 599 of 600 queries and in the top 10 in 600 of 600.
- 5,882 of 5,882 nodes carry `dia_id`, the session index and the message year in `created_at`; captions are kept.
- Missing embeddings stop ingestion with `EmbeddingFailureError` and a count; an empty query vector is handled.
- Categories 1 and 4 are named by their evidence counts, and result files record the judge that ran.
- `retrieval_eval.py` scores evidence recall by `dia_id` (ev@10, ev@50).
- Query-type router: a route with no store now takes every node each store charges. Gold evidence in no store's list fell from 682 of 2,361 (28.9%) to 180 (7.6%); 178 of those are on routes that have a store.
- Tests: `NeuralGraph/tests/test_tesseract_ingestion.py` (17 tests) and `test_tesseract_router.py`; the NeuralGraph suite passes 264.

Still open: the router's scoring (895 of 1,536 questions still score highest on "open"), routing on cross-site questions (no marker fires), latency (a flat scan), and the wiring gap below: no module in `mycelic/` imports Tesseract.

The wiring gap. Tesseract is the routing layer between a neural graph and the Mycelic fabric only in the design. In the code it is wired to nothing:

- No module in `mycelic/` or `mycelic/collective/` imports it. It runs only in `research/benchmarks/` and the routing spike (`research/routing_spike/`).
- The collective guard requires 64 modules to import with the standard library only, and Tesseract needs numpy. The fabric guard forbids `llm_backend` , which Tesseract's package loads eagerly.
- Collective conclusions are written to the collective store only. Nothing publishes them into the fabric's event log.
- The fabric's own retrieval is BM25 with no embeddings and no graph.
- As built, Tesseract scores one session at a time and normalises each store by its own maximum. Merging such scores across stores was measured worse than pooling (H9: -2.5 and -4.4 recall@10).
- One seam already has the right shape: `NeuralGraph/research/coordination` (TesseractCoordinator, NeuralGraphMemoryAdapter). It runs only with test stubs today.

Small-model reader and lab runs

reader-001 and main-001 finished at 05:31 and 05:49 UTC. The real-data reader test scored every model at zero. As far as we can tell, the cause is our post-processing, not the models' reading. R002 then fixed the post-processing (RESULTS run 7): the zero is gone, and every model still reads worse than word matching.

Run	What it tested	Result
reader-001 (R001)	3 models read 150 real NHTSA narratives with the codes hidden	Predicate F1 0.000 [0.000, 0.000] for a-0p5b, a-1p5b and a-4b; lexical baseline 0.434. By the pre-registered rule, each model reads worse

Run	What it tested	Result
main-001	Simulation over 5 seeds, E1 on generator text, G0, E3, X1	Planted patterns found, of 20: X with the model 5 / 9 / 10, codes only 5, lexical 19, each site alone 5 / 10 / 13. The G0 leakage scan with a-4b failed on 1 of 4,257 canaries, later traced to a scanner false alarm and fixed
reader-002 (R002, RESULTS run 7; re-aggregation 37955383933)	The same 3 models read the same 150 narratives, with the post-processing fixed and the raw replies stored	Predicate F1 a-0p5b 0.026 [0.015, 0.038], a-1p5b 0.178 [0.150, 0.209], a-4b 0.218 [0.185, 0.254] from repeat 3 alone (repeats 1 and 2 timed out); lexical baseline 0.434. By the pre-registered rule, every model reads worse
g0-001 (RESULTS run 8)	The G0 canary scan re-run with a-4b in the loop, main-001's settings: device_quality, seed 1, 1,000 synthetic records	0 of 4,259 canaries crossed; positive control 884 hits. Passed
judge-001 (J001, RESULTS run 9)	The same 3 models answer the site verifier's narrow question about the same 150 narratives: one filed and one unfiled component a record, codes hidden	Balanced accuracy a-0p5b 0.500, a-1p5b 0.477 [0.447, 0.503], a-4b 0.387 [0.340, 0.433]; lexical judge 0.670 [0.627, 0.713]; record-blind control 0.527. By the pre-registered rule, every model judges worse

Why R001 is zero.

- **The setup:** the vehicle pack's only entity type is `vehicle`, and the prompt says to copy the vehicle as written.

- **The drop rule:** `ModelExtractor.postprocess` drops an item whole when its type is set but its text is null, or when the text does not resolve to a pack id.
- **The reconstruction,** through the repo's own code: a reader that always types `vehicle` scores 0.000, with 10 of 10 items dropped. The same predicates with both entity fields null score 1.000.
- **The limit:** the raw replies were not stored, so the run itself cannot confirm this. Resolved by R002 (RESULTS run 7): with the drop rule fixed and the replies stored, every model scores above zero but below word matching (predicate F1 0.026, 0.178 and 0.218 against 0.434).

main-001 in brief.

- **X with the model** beat S and model-free R only on narrative-only patterns, which those two cannot see. No seed's lift interval excludes zero.
- **Each site alone** found as many patterns as the collective channel or more, at about five times the alerts.
- **E1 field F1** on generator text was 0.256, 0.527 and 0.707, all below the 0.80 kill line.
- **The G0 failure was a scanner false alarm.** The planted id `TE-002` matched the tail of the site path `g0/region-1/site-002` in HQ's store. It reproduces offline with no model. After the planter learned to avoid the run's own org strings, 1,000-record scans pass on three seeds.

Speed on real text (unit wall time per record, server start included): a-0p5b 9.9–10.5 s, a-1p5b 20.5–21.7 s, a-4b 35.2–38.9 s. Each model ran on a different CPU model, so the three are not directly comparable.

What J001 shows (added after the run; `CHOICE-J001.md`, Runs).

- **The task the product gives a site model fails on public text too,** not only open extraction.
- **The two smaller models do not discriminate:** a-0p5b says "describes the component" on 145 of 150 filed and 145 of 150 unfilled components; a-1p5b on 138 and 131.
- **a-4b discriminates but will not say no:** yes on 97 filed and 37 unfilled; on unfilled components it answers "unclear" 94 times and "no" 19 times. The shipped rule turns unclear into unknown, which is never correct. Counting unclear as a refute would give 0.700, but that reading was chosen after seeing the answers and decides nothing (decision 9).
- **Judge calls are short:** median 1.0 to 1.5 s (a-0p5b), 1.4 to 3.2 s (a-1p5b), 6.0 to 9.0 s (a-4b) on runner CPUs.

Plan for tonight

This is my proposal; edit, reorder or strike anything. The order is: fix the instruments, fix ingestion, then put Tesseract on the path. No new replay is pre-registered until its power check passes; the reading, judging and drafting tests are not replays. Each task ends with its acceptance test.

0. Record what is running (no changes)

- ✓ ~~0.1 Record reader-001 as pre-registered, beside the handicap and the drop counts by reason. Done when: CH0ICE-R001.md and RESULTS Run 6 quote the report verbatim with its sha256.~~
- ✓ ~~0.2 Record main-001 as RESULTS Run 5, whatever it shows. Done when: the same.~~
- ✓ ~~0.3 Trace the G0 canary hit. Done: a scanner false alarm (TE-002 inside site-002), reproduced offline in 8 s. The fix passes the run's org strings to the planter as reserved , with a regression test. The real model re-run passed: g0-001 with a 4b, RESULTS run 8. 0 of 4,259 canaries crossed, and the positive control registered 884 hits.~~

1. Make the measuring instrument honest (offline)

- ☐ **1.1 Fix the chance baseline.** Remove each outcome's own post-window alerts from the shifted null. *Done when:* a world with reactive complaints and no pre-signal gives found $\approx$ expected over 20 seeds, and the saved audits reproduce 20.30 and 6.95.
Status: code done, real-data re-score done, not ticked. DIAG-E002 re-scored the six-make replays in CI. Recalls, X: 14 found, against 30.46 expected (audited) and 16.37 (corrected). Investigations, X: 6 found, against 9.07 and 5.64. The replays sit at chance, not below it. Synthetic checks then showed that the corrected null can err either way. It errs low when unrelated alerts follow an opening, and high when reactions outlast the 26-week post window: there it expects 5.9 finds where none are possible. Its p also counts the observed alignment twice, so it can't go below 2/88. That check failed one of its four pre-registered criteria: a real pre-signal reached $p < 0.05$ in only 5 of 20 seeds, against a bar of 18. Your call (new decision): what replaces this baseline. One option is to leave out shifts within the look-back of zero.
- ☐ **1.2 Add a power check that runs before any replay.** Plant ramps at 0.1, 0.5 and 2.0 complaints a week, with 2 and 4 years of history. *Done when:* the report gives power per channel, and a rule refuses to pre-register a replay below 80% power.
Built, gate fails (merge 2530524, POWER.md run 002). `python -m mycelic.collective.pilot.power` plants ramps and sextuplings on a synthetic background shaped like the vehicle data (60 states). It exits 1 when no channel reaches

80%. At the replays' settings (0.1 a week, 2 years, 26-week look-back), X, S and R_mf found 0 of 30 planted ramps; PRR found 6. No cell passes at a 26-week look-back. At 104 weeks, PRR passes at 0.5 and 2.0 a week over 4 years, but not at 0.1 a week, the signal size DIAG-E001 measured. So a V004 at that size would fail the gate.

- ✓ ~~1.3 Save each channel's alert dates in the audit file. Done when: re-scoring needs only the audit JSON.~~

Done (f019ed7): each channel's `alert_timeline` (available date, week, keys) is saved in `audit.json`.

- ☐ **1.4 Add a pooled national channel and a disproportionality detector** as comparators.
Done when: the pooled channel catches a planted sextupling at least 80% of the time.
Built, target missed (run 001): the pooled channel caught 36 of 60 planted sextuplings (Wilson 0.47–0.71). On the same plants, the state-split X and S caught 8, R_mf 10, and the disproportionality detector PRR 34 (POWER.md). Two causes. The 5-alerts-a-week budget fills every week. And with one pooled site, the high-base-rate penalty hits every common failure. Adapting that rule for a single site is your call, not a tuning step.
- ☐ **1.5 Build vehicle pack v2.** Merge the three old/new name pairs, stop counting a missing reporter as one reporter, and set detector thresholds for 60 sites. *Done when:* every outcome name exists among complaint names, and the null candidate rate at 60 sites is at most 5%.

Built (merge e23f01b, `PACK-V2.md`). The three old/new names now share one predicate. Each complaint counts as its own reporter (an assumption: one complaint, one reporter). The thresholds were tightened: α 0.01 $\rightarrow$ 0.001, minimum sites 2 $\rightarrow$ 3. Null candidate rate per test at 60 sites (synthetic): X 26.15% $\rightarrow$ 0.09%, R_mf 7.01% $\rightarrow$ 0.57%; held-out seeds agree. Not ticked: measured as the share of series that are ever a candidate, X is still above 5% (5.3% and 6.7%). The name check can fail only on the three known old names, because the probe file has no outcome names. No replay has run on pack v2.

2. Fix Tesseract and NeuralGraph ingestion

- ✓ ~~2.1 Add `get_neighbors` to the storage interface and SQLite. Done when: Tesseract on SQLite returns the same top 10 as in memory on 600 queries.~~

Done: on all ten LoCoMo conversations (60 messages each, hash embedder), 600 of 600 queries return the same top 10 on SQLite as in memory.

- ✓ ~~2.2 Fail loudly. A missing embedding raises an error and is counted; an empty query embedding is handled. Done when: a run with no server stops with a clear error.~~

Done: `EmbeddingFailureError` stops ingestion and names how many embeddings are missing. An empty query vector skips the semantic term. Both are tested.

- ☒ **2.3 Keep dia_id, session index and captions; set created_at to the message date;** make the temporal store read the date tokens. *Done when:* evidence recall is scored by dia_id, and 5,882 of 5,882 nodes carry the message year.
Done (merge 921352a): 5,882 of 5,882 nodes carry dia_id and the message year.
`retrieval_eval.py` now scores evidence recall by dia_id (ev@10, ev@50 per category; 9 tests). Hash embedder, all ten conversations: Tesseract ev@10 51.9% → 54.8%, ev@50 67.4% → 72.4% after the router fix below. Plain embedding scores 20.3% / 40.8%.
 These numbers are structural, not retrieval quality.
- ☒ **2.4 Stop a speaker name from swamping the text match.** *Done when:* exact text self-retrieval ranks first at least 99% of the time (cosine: 599 of 600).
Done: exact-text self-retrieval ranks the message first in 599 of 600 queries and in the top 10 in 600 of 600 (all ten conversations, hash embedder). The test's note puts top-1 at 433 of 600 before the fix.
- ☒ **2.5 Fix the LoCoMo category 1/4 swap and record the judge that actually ran.** *Done when:* REPORT, BENCHMARKS and README are restated.
Done: REPORT and BENCHMARKS carry a dated correction (commit afbeed7, merge 61c893e), and README a dated note (commit fd2a67a). The harness maps category 1 to multi_hop and 4 to single_hop, and result files record the judge that ran.
- ☒ **2.6 Write the first Tesseract test suite:** ingestion counts, SQLite, self-retrieval, router, and stable rankings across hash seeds.
Done (merge 921352a): 17 ingestion tests plus 16 router tests. The router sent 922 of 1,536 questions to a route with no store, and each store passed only its top 30-50 candidates. So 682 of 2,361 gold evidence ids (28.9%) were in no store's list. Now a route with no store takes every node each store charges: 180 (7.6%) are left out, 178 of them on routes that do have a store. The NeuralGraph suite passes 264.

3. Put Tesseract on the path to the fabric (needs decision 1 below)

- ☐ **3.1 Write the architecture decision:** Tesseract runs in a site-side process behind the existing Boundary, so the standard-library modules stay that way.
- ☐ **3.2 Build the spike:** a per-site NeuralGraph of narratives → Tesseract through `NeuralGraphMemoryAdapter` → `TesseractCoordinator` with rank fusion and holder selection → conclusions published as fabric events with lineage. *Done when:* on a planted multi-site world, routed questioning finds more planted patterns than random holder selection, with a 95% interval above zero.
Built as a spike, not ticked (`docs/collective/ROUTING-SPIKE.md`, `research/routing_spike/`; it informs decisions 1 and 2 and decides neither). Each site runs a NeuralGraph of its own records, ranked by Tesseract behind the Boundary. HQ

sees only k-suppressed cells, the detector candidate and a fixed descriptor. It routes each question to 3 of 6 sites, and conclusions go to the fabric with lineage to the site verdicts. The setup is synthetic, 20 seeds, with the design and criterion committed before any code. Results:

- **Run 1 passed, but trivially:** routed minus random was +19.4525 patterns found [18.5675, 20.4275]. Routing on the key's own cells alone (R1) does as well. HQ routing without those cells (R0, cell-blind) gains only +1.45 over random (Run 2: +1.545). So nearly all of the gain is the key's own cells marking the holders. Tesseract ranks only inside each site; its scores never leave the site.
- **Run 2, after review found the site ranking saw records dated after the question, fails:** the primary still passes at +22.195 [21.2825, 23.1475]. The no-plant control misses its bar, false conclusions $R - U + 0.0349$ [0.0166, 0.0554] against ≤ 0.05 .
- **For decision 1:** the separate-process mode works; 360 of 360 verdicts were byte-identical to in-process.
- **The entity defect, fixed but not yet run:** the Tesseract site ranking ignored the entity. So a site could miss the records its own cells counted, and then contest a true key. Since commit 9aa8230 a site reads the shipped entity-matched records first and fills the rest, up to 50, with Tesseract's. No Run 3 has used it. Run 2's prereg refuses the changed code, so a Run 3 needs a new prereg; whether to run one is your call.

- ☐ **3.3 Define what the router receives** when there is no user question: a detector candidate, or an HQ question.

4. Reader

- ☒ ~~**4.1 Pre-register R002, then re-run the reader test on the same 150 records (reader-001 scored zero through the post-processing).**~~ Attach predicates to the structured vehicle and keep raw replies for public data.

Status: rule committed (CHOICE-R002.md). R002 changes only the post-processing and keeps the raw replies, so a change in score has one cause. The rest of the first scope is left for R003, the next item.

Result (RESULTS run 7, CHOICE-R002.md). Predicate F1 against the filed codes: a-0p5b 0.026 [0.015, 0.038], a-1p5b 0.178 [0.150, 0.209], a-4b 0.218 [0.185, 0.254]. The a-4b score comes from one repeat; two timed out at 150 minutes. The lexical extractor scores 0.434. By the rule, every model reads worse. The fix removed the zero, and the raw replies are now stored. The models name many components per complaint (a-1p5b re-attached 7,513 items over 3×150 records), and most don't match the filed ones. The

scores came from re-aggregating the run's artifacts in CI, declared before it ran: the run's own aggregate skipped scoring when the reference model's repeats were incomplete.

- ☐ **4.1, the rest: an R003 rule (open, your call).** Merge the name pairs, drop the `unknown_or_other` trap, define negation, and use one repeat at temperature 0. R002 left these out.
- ☒ **4.2 Put drops by reason and the zero-claim share into the lab report.**
Done (merge a230de2): drops by reason, re-attachments and the zero-claim share appear in E1's comparison, the lab aggregate and the run summary.
- ☒ **4.3 Judge test J001** (added during the night): the three models answer the site verifier's narrow question about the same 150 records. *Result:* every model judges worse than the lexical judge (RESULTS run 9, `CH0ICE-J001.md`). The rule was amended twice before the run, after two reviews found the question draw leaking the label and the first control biased low.
- ☒ **4.4 Pack drafter test D001/D002** (added during the night): can a pack be drafted from an export alone, in a field no pack covers? *Result:* D001 crashed at the MSHA split on quoted fields (its result). D002, the same rule with quoted fields parsed and the run starting after the splits, passed all six criteria: MSHA 0.559 against 0.344; NHTSA 0.590 against the hand pack's 0.406 (`docs/collective/onboard/CH0ICE-D002.md`).

5. Docs honesty

- ☐ **5.1 Restate the replay verdict** as "uninformative: lexical counts and model-free detectors only". Withdraw "public data is spent". Say on the site that the audit reads lexically today, and credit the README's +8.9 to owner routing.

After the gates (not tonight):

- ☐ **V004.** A replay with 2020–2024 history, each defect's earliest action as the outcome and an 8-quarter look-back. It runs pooled and per-state, with and without the questioning step, on GitHub runners, because NHTSA is blocked from this sandbox. It has no model channel: R002 did not pass. Its null is to be decided under 1.1, since the corrected one's bias runs in an unknown direction. At DIAG-E001's 0.1 a week the rule refuses to pre-register it. With four years of history and this look-back, the best channel found 15 of 30 planted ramps (PRR, `POWER.md`).

Decisions for the chief scientist

These change what the code is allowed to do or what counts as a result, so I should not make them alone. Pick a call in the last column; decision 1 gates block 3 of the plan. In the PDF the last column shows only a small ▼ marker: write the choice (Agree, Change or Discuss) in that cell, or pick it in the shared doc.

#	Decision	Options	My recommendation	Your call
1	Where Tesseract runs at a site, given the stdlib-only and fabric import guards	(a) A separate site-side process behind the existing Boundary; (b) lazy imports plus a narrow guard exemption	(a): the guarantees the guards protect stay intact	▼
2	What Tesseract routes when nobody asks a question	Detector candidates sent to the sites that hold evidence; HQ analyst questions; both	Candidates to holder sites. Owner routing is the only routing with a measured gain (+7.4 recall@10). Test it against random holder choice	▼
3	The replay endpoint	Keep the 26-week look-back; or use 8 quarters and each defect's earliest action (investigation or recall)	8 quarters and earliest action, declaring that DIAG-E001 was seen first. At the measured 0.1 a week, though, the rule refuses to pre-register V004 (POWER.md: best channel 15 of 30). No model channel (R002 did not pass); the null is open under 1.1	▼
4	Is a US state a fair stand-in for a company site?	State split only; pooled national only; both	Both, with the pooled national test as the comparator the state split must beat	▼

#	Decision	Options	My recommendation	Your call
5	The Dec-2025 Tesseract numbers and the README's +8.9	Keep; retire and credit owner routing	Retire. The Dec-2025 harness routed on the gold label, and +8.9 is owner routing	▼
6	"Public data is spent" in MARKET and RESULTS	Keep; withdraw until V004	Withdraw: the replays could not have seen a signal of the measured size	▼
7	The embedding model for Tesseract tests	Hash embedder only; a real embedding model on a CI runner	Hash embedder for structural tests; a real model on CI for any headline number	▼
8	Must a site model name the entity, or only read predicates about entities the site's systems already know?	Name the entity; predicates only	Predicates only where the entity is structured (vehicles). R002 already ran this way: its rule 2 attaches predicates to the record's structured vehicle. Please ratify that, or reverse it for later tests	▼
9	How a site verifier's "unclear" counts (J001: a-4b answered unclear on 94 of 150 unfilled components)	Keep it as unknown, as shipped; count it as a refute; ask once more	Decide before any new judge test, and test the choice only on records these models have not judged. Never re-score J001	▼

Open research questions

Each question below can move the verdict. The first two decide whether the problem is our pipeline or the data.

#	Question	Why it matters	How we answer it
Q1	Do public complaints carry a pre-event signal any detector could see?	DIAG-E001 gives only means: 261 of 589 recalls dated 2023 had any matching complaint before them	The per-event distribution from DIAG-E001: the share of events above 0.5 complaints a week sets the power ceiling
Q2	Did the pipeline or the data cause the null?	It decides whether to fix detectors or move to company data	Pooled national and disproportionality detectors on the same exports (task 1.4), against a baseline that replaces the corrected one, whose bias runs in an unknown direction (task 1.1, open)
Q3	How many post-outcome alerts are reactive, and how many are real rises detected late?	Post/pre alert ratios are 4.39, 3.43 and 2.73. Late detection is fixable; reaction is not	Alert dates per channel, now saved (task 1.3, done) but not yet analysed, plus NHTSA's own recall-complaint flag in the raw file
Q4	Does Tesseract add anything over plain embedding once owner routing is applied?	With owner routing, embedding ties Tesseract on category 1 and beats it by 2.5 recall@10 points on category 4	Embedding-local plus pairs against Tesseract-local plus pairs, end to end, on the same 282 question ids
Q5	Does owner routing still work with more than 2 holders and questions that name no holder?	That is the cross-site situation. On LoCoMo, 0% of category-1 gold is spoken by the other agent, so routing there is close to tautological	Task 3.2's spike did not settle it: its holders always leave cells, and those cells did the routing. Still needed: holders whose evidence the cells do not mark, read by a model

#	Question	Why it matters	How we answer it
Q6	Does the questioning step add finds on real data?	The simulation credits it with most discovery (0.625 → 0.080 without it). As built, it can only confirm or drop detector candidates, not add finds	V004 with and without the questioning step. Descent to sites that raised no alert must be built first
Q7	Can small models read real narratives better than word-matching, once the handicap is removed?	It is the "models at the site" premise	Answered, negative on public text: in R002 (task 4.1) every model read worse than word matching on the same 150 records (RESULTS run 7). Company text is untested
Q8	Would a company's own records carry 10–20× NHTSA's volume per vehicle and component?	At a 26-week look-back no channel reaches 80% even at 2.0 a week, 20× the measured public signal; at 104 weeks with 4 years of history, PRR passes at 0.5 a week, not at 0.1 (POWER.md)	Discovery calls and a pilot export; this is a founder task
Q9	How many tokens is a real record?	Latency and cost assume 24 tokens. The extraction prompt alone is 2,768 characters, and NHTSA records reach 2,824	The measured <code>tokens_in</code> from the lab ledgers of reader-001, reader-002 (R002) and judge-001 (J001), not yet read out
Q10	Is J001's result a property of the prompt and verdict rule, or of the models?	It decides whether to change the verifier or give up on site models for judging	A new choice file on a fresh draw: unclear's meaning fixed in advance (decision 9), with the codes present as the product sends them (J001 trap 4) and without

Sources

All paths are in `anovruzov/NeuralGraph`, branch `claude/mycelic-production-deployment-r93cez`. A copy of this handoff is committed at `docs/handoff/HANDOFF-2026-10-09.md`.

- **Rules and results:** `docs/collective/replay/vehicles/` (`CHOICE-V001.md`, `CHOICE-V002.md`, `CHOICE-V003.md`, `CHOICE-R001.md`, `CHOICE-R002.md`, `CHOICE-J001.md`, `DIAG-E001.md`, `DIAG-E002.md`, `PACK-V2.md`); `docs/collective/` (`POWER.md`, `PILOT.md`, `ROUTING-SPIKE.md`); `docs/collective/onboard/` (`CHOICE-D001.md`, `CHOICE-D002.md`); `docs/lab/RESULTS.md`; `docs/strategy/MARKET.md` section 5.1.
- **Runs:** V001 run 002 [37870255727](#); V002 [37871216150](#); V003 [37871956063](#); DIAG-E001 run 2 [37883179901](#); reader-001 [37874950202](#); main-001 [37866392839](#); DIAG-E002 run 5 [37911451566](#) and run 6 [37920746888](#); reader-002 [37910989965](#), re-aggregated in [37955383933](#); g0-001 [37938605933](#); judge-001 [37996675559](#); D001 [38017322304](#); D002 [38024536763](#).
- **Collective pipeline:** `mycelic/collective/pilot/audit.py`, `evaluate/baselines.py`, `edge/site.py`, `edge/extract.py`, `detect/detectors.py`, `experiments/openfda_replay.py`; `tools/market/nhtsa_export.py`, `vehicle_pack.py`; `tests/mycelic/test_collective_guards.py`.
- **Tesseract and NeuralGraph:** `NeuralGraph/research/retrieval/tesseract.py`; `NeuralGraph/research/coordination/`; `research/benchmarks/runner.py`; `research/reports/AUDIT.md`, `H9_two_agent.md`, `H4_H6_scoring.md`, `REPORT.md`; `docs/BENCHMARKS.md` (evidence recall by `dia_id`).
- **Fabric:** `mycelic/retrieval.py`, `mycelic/aggregation.py`, `mycelic/service.py`; `docs/MYCELIC_ENTERPRISE.md` (simulation ablations).
- **Method:** six code investigations, each finding re-checked by an independent reviewer against the code, the saved audits and simulations through the repo's own pipeline. Offline Tesseract numbers use the hash embedder.